PYTHON · EXCEPCIONES · ARCHIVOS · PERSISTENCIA · POO

Laboratorio 4

Solución desarrollada y respuestas de análisis

Gestor de inventario persistente con JSON y exportación CSV

Aprendiz	Yohan Fernando Cuellar Ospino
Ficha	3388087
Repositorio personal	[Pega el enlace directo de GitHub]
Fecha	23-10-2026

1. Estado y alcance

Se resolvieron las actividades principales de las prácticas 4.1 a 4.4 mediante explicaciones, resultados esperados y ejemplos de código. La práctica 4.5 se implementó en **gestor_inventario/** como una aplicación de consola dividida en módulos, con validaciones, carga segura de JSON, guardado, exportación CSV y pruebas automatizadas.

- El producto valida código, nombre, categoría, precio numérico, finitud del precio y cantidad entera no negativa.
- El inventario registra por código, busca, lista, actualiza, elimina, consulta agotados y calcula resumen.
- La carga de JSON valida todos los registros en un inventario temporal; ante cualquier error no entrega una carga parcial ni sobrescribe el archivo fuente.
- El guardado escribe un archivo temporal y lo reemplaza al terminar la escritura.
- El CSV incluye encabezados y calcula el valor por fila como precio por cantidad.

Limitación de la entrega: no fue posible copiar ni comparar el gestor del Laboratorio 3 porque solo se

proporcionó el enunciado del Laboratorio 4. La migración a partir de la versión anterior debe ajustarse a sus clases y nombres reales.

2. Práctica 4.1: excepciones y validaciones

Conversión y edad válida

Se atiende `ValueError` solo alrededor de la conversión. El rango se comprueba después con condicionales: convertir correctamente no implica que el dato cumpla la regla del problema.

```
try:
    edad = int(input("Edad: "))
except ValueError:
    print("La edad debe ser un número entero.")
else:
    if 0 <= edad <= 120:
        print("Edad aceptada:", edad)
    else:
        print("La edad debe estar entre 0 y 120.")
```

Casos: `veinte` produce error de conversión; `-1` y `121` están fuera del rango; `20` se acepta.

Reintento de precio

El ciclo vuelve a preguntar ante texto inválido o un valor no positivo. En el reto integrador se refuerza la regla usando `math.isfinite()`, porque `nan` e `inf` también pueden convertirse a `float`.

```
import math

while True:
    try:
        precio = float(input("Precio: "))
    except ValueError:
        print("Debes escribir un número.")
        continue
    if not math.isfinite(precio) or precio <= 0:
        print("El precio debe ser positivo y finito.")
        continue
    print("Precio aceptado:", precio)
    break
```

División segura y depuración

```
try:
    dividendo = float(input("Dividendo: "))
    divisor = float(input("Divisor: "))
except ValueError:
    print("La entrada no puede convertirse a número.")
else:
    if divisor == 0:
        print("No se puede dividir entre cero.")
    else:
        print("Resultado:", dividendo / divisor)
```

El bloque de depuración que captura `ZeroDivisionError` no atiende `int("hola")`, porque esa conversión lanza `ValueError`. Se debe capturar el error que realmente se espera, no uno genérico que oculte defectos.

Estado seguro y compras

Una operación no debe asignar primero una cantidad y comprobarla después. La validación ocurre antes de modificar: el método `actualizar_cantidad` rechaza tipos incorrectos y cantidades negativas antes de tocar `_cantidad`.

```
compras = []
while True:
    try:
        importe = float(input("Importe (0 termina): "))
    except ValueError:
        print("Escribe un importe numérico.")
        continue
    if importe == 0:
        break
    if not math.isfinite(importe) or importe < 0:
        print("El importe debe ser positivo y finito, o cero para terminar.")
        continue
    compras.append(importe)
print("Compras aceptadas:", len(compras))
print("Total:", sum(compras))
```

Respuestas de concepto: `raise` comunica el incumplimiento de una regla; `else` se ejecuta si el `try` termina sin excepción; `finally` se ejecuta al salir normalmente del bloque haya error o no. Se usa `if` para reglas de negocio y `try/except` para operaciones que pueden lanzar una excepción esperada.

Excepciones concretas y cierre

`int("abc")` produce `ValueError`; `int(None)` produce `TypeError`. No se captura `TypeError` para resolver una entrada mal escrita si la causa real es `ValueError`. Para el ejercicio del archivo inexistente:

```
from pathlib import Path

ruta = Path("archivo_que_no_existe.txt")
try:
    contenido = ruta.read_text(encoding="utf-8")
except FileNotFoundError:
    print("No se encontró el archivo solicitado.")
else:
    print(contenido)
```

Un bloque de lectura con `finally` imprime su mensaje de cierre tanto si la conversión funcionó como si se atendió el error. Para el constructor de producto se rechaza el dato antes de asignarlo:

```
class ProductoValidado:
    def __init__(self, nombre, precio):
        if not nombre.strip():
            raise ValueError("El nombre es obligatorio.")
        if precio <= 0:
            raise ValueError("El precio debe ser positivo.")
        self.nombre = nombre
        self.precio = precio
```

3. Práctica 4.2: archivos y rutas

Bitácora persistente

La bitácora agrega una línea en modo `a`; así conserva ejecuciones anteriores. Se fija la ruta a partir de `__file__` para que no dependa del directorio actual de la terminal.

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
RUTA = BASE_DIR / "pruebas.txt"

prueba = input("Nombre de la prueba: ").strip()
resultado = input("Resultado (aprobada/fallida): ").strip()
observacion = input("Observación: ").strip()
with RUTA.open("a", encoding="utf-8") as archivo:
    archivo.write(f"{prueba} | {resultado} | {observacion}\n")
```

Reporte de texto

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
carpeta = BASE_DIR / "reportes"
carpeta.mkdir(parents=True, exist_ok=True)
ruta = carpeta / "resumen.txt"
ruta.write_text("Productos: 3\nUnidades: 6\nValor: 1740000\n", encoding="utf-8")
print(ruta.read_text(encoding="utf-8"))
print("Guardado en:", ruta.resolve())
```

Se usa `with` para cerrar el archivo incluso si ocurre una excepción. `write_text()` en modo de escritura reemplaza su contenido; por eso escribir “Registro 1” y después “Registro 2” deja solo el segundo. Para conservar líneas anteriores se usa `open(..., "a")`.

Notas de texto, líneas e historial

Para escribir y leer notas sin cerrar manualmente los archivos:

```
with open("notas.txt", "w", encoding="utf-8") as archivo:
    archivo.write("Primera observación\nSegunda observación\n")
with open("notas.txt", "r", encoding="utf-8") as archivo:
    for numero, linea in enumerate(archivo, start=1):
        print(numero, "-", linea.strip())
```

El reto de historial usa el mismo patrón que la bitácora, añadiendo el valor y protegiendo la consulta si no existe el archivo:

```
from pathlib import Path

ruta = Path(__file__).resolve().parent / "operaciones.txt"
opcion = ""
while opcion != "0":
    opcion = input("1 Registrar | 2 Mostrar | 0 Salir: ").strip()
    if opcion == "1":
        concepto = input("Concepto: ").strip()
        valor = input("Valor: ").strip()
        with ruta.open("a", encoding="utf-8") as archivo:
            archivo.write(f"{concepto} | {valor}\n")
    elif opcion == "2":
        try:
            print(ruta.read_text(encoding="utf-8"))
        except FileNotFoundError:
            print("Todavía no hay operaciones registradas.")
```

Historial de operaciones

La solución mantiene un menú de registrar, consultar y salir. Cada operación se agrega a un archivo de texto con codificación UTF-8; al consultar se captura `FileNotFoundError` y se comunica que aún no hay registros. En el proyecto integrador, la misma necesidad se resuelve con JSON porque el estado está compuesto por registros estructurados.

4. Práctica 4.3: JSON y CSV

Serialización de objetos y catálogo

Un objeto de una clase propia no se serializa directamente con `json.dump()`. `Producto.a_diccionario()` transforma sus atributos en tipos compatibles con JSON; `Producto.desde_diccionario()` reconstruye la instancia utilizando el constructor y sus validaciones. Para el ejercicio de libros se aplica el mismo patrón con una lista de diccionarios que contenga código, título, autor y año.

```
datos = [
    {"codigo": "L001", "titulo": "Python inicial", "autor": "A. Ruiz", "anio": 2024},
    {"codigo": "L002", "titulo": "Datos claros", "autor": "M. Sol", "anio": 2023},
    {"codigo": "L003", "titulo": "Objetos", "autor": "C. Vega", "anio": 2022},
]
with open("libros.json", "w", encoding="utf-8") as archivo:
    json.dump(datos, archivo, ensure_ascii=False, indent=4)
```

La lectura del catálogo vuelve a construir los diccionarios y muestra sus campos:

```
with open("libros.json", "r", encoding="utf-8") as archivo:
    libros = json.load(archivo)
for libro in libros:
    print(libro["codigo"], libro["titulo"], libro["autor"], libro["anio"])
```

Lectura protegida

Un archivo inexistente puede representar la primera ejecución y se inicia una lista vacía. Un error JSON o una estructura inválida no significa que el archivo sea descartable: se informa el problema y no se escribe encima. La guía de código mantiene separados esos estados.

Ventas CSV

Se escribe con `csv.DictWriter`, `newline=""` y UTF-8. Al leer con `DictReader`, los campos son cadenas y deben convertirse antes de sumarlos.

```
import csv

ventas = [
    {"codigo": "V01", "producto": "Teclado", "cantidad": 2, "total": 170000},
    {"codigo": "V02", "producto": "Mouse", "cantidad": 1, "total": 45000},
    {"codigo": "V03", "producto": "Monitor", "cantidad": 1, "total": 700000},
    {"codigo": "V04", "producto": "Cable", "cantidad": 3, "total": 60000},
]
with open("ventas.csv", "w", newline="", encoding="utf-8") as archivo:
    campos = ["codigo", "producto", "cantidad", "total"]
    escritor = csv.DictWriter(archivo, fieldnames=campos)
    escritor.writeheader()
    escritor.writerows(ventas)
with open("ventas.csv", newline="", encoding="utf-8") as archivo:
    total_vendido = sum(float(fila["total"]) for fila in csv.DictReader(archivo))
print(total_vendido)  # 975000.0
```

El JSON conserva la estructura necesaria para reconstruir la aplicación; el CSV es una exportación tabular para hojas de cálculo. El JSON de depuración dado carece de una coma entre objetos y debe corregirse agregándola después del primer objeto.

Importación protegida y exportación tabular

Una lectura mínima segura distingue la primera ejecución de un documento existente pero dañado. En el gestor final también se valida el tipo de la raíz y cada registro antes de aceptar el inventario.

```
try:
    with open("productos.json", encoding="utf-8") as archivo:
        productos = json.load(archivo)
except FileNotFoundError:
    productos = []
except json.JSONDecodeError as error:
    print(f"JSON inválido en línea {error.lineno}; no se sobrescribió.")
else:
    if not isinstance(productos, list):
        print("La estructura debe ser una lista; no se sobrescribió.")
    else:
        print("Registros leídos:", len(productos))
```

La exportación de ventas del ejercicio se puede comprobar releyendo el CSV, convirtiendo el campo de total a número y sumando el resultado. Si una columna representa precio por existencias, se multiplica precio por cantidad, no se copia el precio unitario.

5. Práctica 4.4: POO intermedia

Herencia y sobrescritura

`Instructor` es un tipo de `Empleado`, por lo que heredar y llamar `super().__init__(nombre)` es razonable. El ejercicio de vehículos se resuelve análogamente: `Motocicleta(Vehiculo)` inicializa marca y modelo mediante `super()` y agrega cilindraje.

Polimorfismo en reportes y notificaciones

```
class ReporteTexto:
    def generar(self, datos):
        return "Registros: " + ", ".join(str(dato) for dato in datos)

class ReporteResumen:
    def generar(self, datos):
        return f"Cantidad de registros: {len(datos)}"

reportes = [ReporteTexto(), ReporteResumen()]
datos = ["Teclado", "Mouse"]
for reporte in reportes:
    print(reporte.generar(datos))
```

En el reto de notificaciones, `NotificacionCorreo` y `NotificacionConsola` implementan `enviar(mensaje)` de forma distinta. El ciclo puede llamar al mismo método sin decidir el tipo concreto. El ejemplo de depuración falla porque el constructor de `Aprendiz` nunca inicializa `Persona`; la corrección es `super().__init__(nombre)`. En el gestor, inventario contiene productos: es composición, no una relación “es-un”, y no se justifica una herencia artificial.

```
class Vehiculo:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

class Motocicleta(Vehiculo):
    def __init__(self, marca, modelo, cilindraje):
        super().__init__(marca, modelo)
        self.cilindraje = cilindraje

class NotificacionCorreo:
    def enviar(self, mensaje):
        return f"Correo enviado: {mensaje}"

class NotificacionConsola:
    def enviar(self, mensaje):
```

```
        return f"Consola: {mensaje}"

for notificacion in (NotificacionCorreo(), NotificacionConsola()):
    print(notificacion.enviar("Inventario actualizado"))
```

6. Práctica 4.5: solución integradora

producto.py	Validar atributos, actualizar cantidad, convertir a diccionario y reconstruir.
inventario.py	Administrar productos por código, evitar duplicados, consultar y resumir.
almacenamiento.py	Resolver la ruta desde el módulo, cargar JSON sin parciales y guardar con temporal.
reportes.py	Exportar el CSV con valor de inventario por producto.
main.py	Menú, entrada y salida de datos, coordinación y mensajes de error.

La aplicación rechaza códigos repetidos, nombres vacíos, categorías desconocidas, precios no positivos o no finitos y cantidades negativas o no enteras. Un precio cero, negativo, `nan` o `inf` no se acepta; una existencia igual a cero sí.

Resultados de la prueba numérica

P001	Teclado	85000	4	340000
P002	Mouse	45000	0	0
P003	Monitor	700000	2	1400000

Inicial: 3 productos, 6 unidades, valor $1.740.000 y 2 categorías. El Mouse aparece sin existencias. Al cambiar su cantidad a 5: 11 unidades y valor $1.965.000. La celda del CSV de P002 en `valor_inventario` vale 225000.0.

Pruebas ejecutadas

Comando: `python -m unittest discover -s tests -v`, desde `gestor_inventario/`. Resultado observado: 4 pruebas aprobadas.

Rechazo de precios inválidos y cantidad negativa	Aprobada
Rechazo de código duplicado	Aprobada
JSON dañado intacto y duplicados sin carga parcial	Aprobada
Guardar, recuperar, calcular resumen y exportar CSV	Aprobada

También se ejecutó el menú con JSON inexistente y selección 0: inició con cero productos y terminó normalmente. Para ejecutar manualmente, desde la carpeta del proyecto: `python main.py`. Para repetir pruebas: `python -m unittest discover -s tests -v`.

7. Análisis situacional

Pregunta 1. El programa se cerró y los productos ya no aparecen

Qué haría: comprobaría primero si la operación de guardado terminó correctamente y si el archivo esperado existe. **Por qué:** los cambios en memoria desaparecen al terminar el proceso y una ruta relativa puede haber escrito en otra carpeta. **Cómo:** revisaría el mensaje de confirmación, la ruta que imprime la aplicación y `RUTA_JSON`, construida desde `Path(__file__).resolve().parent`; inspeccionaría el JSON sin modificarlo. **Comprobación:** registraría un producto, confirmaría el archivo y sus datos, cerraría el proceso, lo iniciaría de nuevo y verificaría que aparece en el listado y en el resumen.

Pregunta 2. JSON existente con una coma faltante

Qué haría: conservaría una copia, leería el mensaje y la ubicación del error, y corregiría el JSON o restauraría una copia válida. **Por qué:** reemplazarlo por una lista vacía destruiría datos que quizá se puedan recuperar. **Cómo:** capturaría `JSONDecodeError`, informaría línea y columna, bloquearía las modificaciones y no llamaría al guardado automático. **Comprobación:** compararía el archivo dañado antes y después para confirmar que no cambió; luego restauraría el respaldo y verificaría la carga completa.

Pregunta 3. Hay dos productos con el mismo código

Qué haría: aplicaría la unicidad en la operación de registro y también durante la reconstrucción del JSON. **Por qué:** validar solo el formulario deja pasar duplicados introducidos por edición externa o por un archivo antiguo. **Cómo:** el inventario usa el código como clave única y carga cada registro en una instancia temporal; si uno falla, esa instancia se descarta. **Comprobación:** prepararía un JSON con códigos repetidos, intentaría cargarlo y comprobaría que se rechaza el conjunto completo y que el archivo original queda intacto.

Pregunta 4. CSV presenta el precio unitario como valor total

Qué haría: revisaría el cálculo que genera la columna `valor_inventario` en `reportes.py`. **Por qué:** el valor de existencias no es el precio unitario sino el precio multiplicado por la cantidad. **Cómo:** escribiría `precio * cantidad` para cada fila, sin alterar las existencias del inventario. **Comprobación:** probaría con P002 (45000 por 5) y comprobaría 225000.0 tanto en el CSV como al leerlo con `csv.DictReader`; también compararía el total con datos conocidos.

Pregunta 5. Una corrección rompe la carga después de modularizar

Qué haría: identificaría el cambio exacto mediante el historial de commits y lo reproduciría con el mismo JSON y comando que fallan. **Por qué:** el historial permite acotar qué modificación alteró el contrato entre módulos, sin revertir cambios no relacionados. **Cómo:** compararía el commit anterior y posterior, revisaría firmas, imports, rutas y excepciones en la frontera entre `main.py` y `almacenamiento.py`, y prepararía una prueba de regresión. **Comprobación:** ejecutaría dicha prueba con archivo válido, inexistente y dañado, además de toda la suite. Documentaría el commit real, la corrección y el resultado; no se incluyen hashes ficticios en esta entrega.

blicar los cambios. El workspace recibido no tenía configuración Git ni commits para verificar.

Comparación del avance

Según la guía, el Laboratorio 3 ya organizaba productos e inventario mediante funciones, módulos y clases, pero perdía los datos al cerrar. Este laboratorio añade validaciones y manejo de errores, rutas estables, serialización JSON, reconstrucción validada y exportación CSV. Los laboratorios 1 y 2 deben describirse de acuerdo con los proyectos reales del aprendiz; no se proporcionaron sus materiales, así que no atribuyo funcionalidades específicas que no pueda comprobar.